

Relatório Técnico: Implementação de Padrões de Teste (Test Patterns)

Data: 31 de Maio de 2026

Escopo: Serviço de Checkout de E-commerce

Aluno: André Augusto Silva Carvalho

1. Introdução

Este relatório documenta a implementação de Padrões de Teste (Test Patterns) aplicados a um serviço de checkout de e-commerce. O trabalho abrange dois grupos de padrões: Padrões de Criação de Dados (Object Mother e Data Builder) e Padrões de Test Doubles (Stubs e Mocks). O objetivo central é demonstrar como esses padrões previnem Test Smells e contribuem para uma suíte de testes legível, isolada e sustentável.

2. Padrões de Criação de Dados (Builders)

2.1 Por que CarrinhoBuilder e não CarrinhoMother?

O padrão Object Mother é ideal para entidades simples e fixas. No projeto, ele foi aplicado à entidade User, que possui apenas dois estados relevantes para os testes: PADRÃO e PREMIUM. Esses estados raramente variam entre cenários, tornando métodos estáticos como UserMother.umUsuarioPadrao() suficientes e expressivos.

A entidade Carrinho, por outro lado, é intrinsecamente complexa: pode pertencer a diferentes usuários, conter zero ou muitos itens com valores distintos e representar cenários radicalmente diferentes entre os testes. Aplicar o Object Mother ao Carrinho exigiria uma explosão de métodos como umCarrinhoVazioDeUsuarioPremiumComUmItem, tornando o código impossível de manter. O padrão Data Builder resolve esse problema ao oferecer uma API fluente e composável, onde cada teste configura apenas o que importa para seu cenário específico.

2.2 Exemplo: Antes e Depois do Data Builder

O trecho a seguir ilustra como era o setup manual (antes) e como ficou com o CarrinhoBuilder (depois).

Antes — Setup manual obscuro:

```
// Setup manual: o leitor precisa entender cada detalhe
const user = new User(2, 'Maria Premium', 'premium@email.com',
```

```
'PREMIUM');  
const item1 = new Item('Produto A', 100);  
const item2 = new Item('Produto B', 100);  
const carrinho = new Carrinho(user, [item1, item2]);  
// Qual é o objetivo deste carrinho? Não está claro pelo setup.
```

Depois — Setup com CarrinhoBuilder:

```
// Com o Builder: intenção declarada, setup mínimo e legível  
const carrinho = new CarrinhoBuilder()  
  .comUser(UserMother.umUsuarioPremium()) // usuário PREMIUM  
  .comItens([new Item('Produto A', 100), new Item('Produto B',  
100)]) // total = R$200  
  .build();
```

2.3 Como o Builder melhora legibilidade e manutenção

O padrão Data Builder melhora os testes em três dimensões fundamentais:

- **Legibilidade:** Cada chamada de método descreve uma intenção (.comUser(...), .vazio()), tornando o teste autoexplicativo sem a necessidade de comentários.
- **Manutenção:** Se a assinatura do construtor de Carrinho mudar, somente o Builder precisa ser atualizado — todos os testes permanecem intactos.
- **Foco:** Ao fornecer valores padrão sensatos no construtor, o Builder garante que cada teste declare apenas o que é relevante para seu cenário, eliminando o Test Smell de Setup Obscuro (Obscure Setup).

3. Padrões de Test Doubles (Mocks vs. Stubs)

3.1 O teste analisado: cliente Premium com sucesso

O cenário escolhido para análise é o teste "quando um cliente Premium finaliza a compra — deve aplicar 10% de desconto e enviar e-mail de confirmação". Nele, três dependências externas são substituídas por dublês de teste: GatewayPagamento, PedidoRepository e EmailService.

```
const gatewayStub = { cobrar: jest.fn().mockResolvedValue({ success:  
true }) };  
const repositoryStub = { salvar:  
jest.fn().mockResolvedValue(pedidoFake) };
```

```
const emailMock = { enviarEmail:
jest.fn().mockResolvedValue(undefined) };

// Assert – Verificação de Comportamento:
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, CARTAO_DUMMY);
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com', 'Seu Pedido foi Aprovado!', '...'
);
```

3.2 Identificando Stubs e Mocks no teste

Dependência	Tipo	Papel no teste
GatewayPagamento	Stub	Fornece retorno controlado ({ success: true }) para que o fluxo chegue ao assert sem depender do gateway real.
PedidoRepository	Stub	Devolve um pedido falso já persistido (pedidoFake), permitindo verificar o valor de retorno do método.
EmailService	Mock	Registra as chamadas recebidas para que possamos verificar SE foi chamado, QUANTAS vezes e COM QUAIS argumentos.

3.3 Por que GatewayPagamento foi (principalmente) um Stub?

O GatewayPagamento é um Stub porque o que nos interessa neste teste não é verificar se ele foi chamado, mas sim o estado resultante do sistema após a chamada. Queremos saber: dado que o pagamento foi aprovado, o CheckoutService calculou o desconto corretamente?

Retornou o pedido persistido?

O papel do GatewayStub é apenas controlar o fluxo — garantindo que o checkout não retorne null por falha de pagamento. Embora o teste também verifique o argumento passado ao gateway (expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, CARTAO_DUMMY)), essa verificação serve para confirmar que a regra de negócio (desconto de 10%) foi aplicada corretamente ao valor, o que é uma forma de Verificação de Estado indireta. A identidade do

gateway em si não é o objeto do teste.

3.4 Por que EmailService foi um Mock?

O EmailService é um Mock porque seu envio de e-mail é um efeito colateral observável — um comportamento que o sistema deve produzir, mas que não retorna um valor utilizável. Em termos de Martin Fowler (Mocks Aren't Stubs, 2007): Mocks são usados para verificar interações.

Não há estado visível a checar após o envio do e-mail. O próprio CheckoutService captura exceções do EmailService sem propagá-las. Portanto, a única forma de garantir que o e-mail foi enviado corretamente é pela Verificação de Comportamento: confirmar que enviarEmail foi chamado exatamente uma vez, com o destinatário, assunto e corpo corretos. Isso é o que os assertions toHaveBeenCalledTimes(1) e toHaveBeenCalledWith(...) garantem.

4. Conclusão

O uso deliberado de Padrões de Teste transforma a suíte de testes de um conjunto frágil e difícil de manter em uma documentação viva e confiável do comportamento do sistema. Os Padrões de Criação de Dados — Object Mother e Data Builder — eliminam o Test Smell de Setup Obscuro. O Object Mother centraliza instâncias fixas e reutilizáveis de entidades simples, enquanto o Data Builder oferece uma API fluente para cenários complexos, tornando explícita a intenção de cada teste sem expor detalhes de construção irrelevantes. O resultado é um código de teste que se lê como uma especificação de negócio.

Os Padrões de Test Doubles — Stubs e Mocks — são complementares e servem a propósitos distintos. Stubs controlam o fluxo ao fornecer respostas pré-definidas para dependências externas, permitindo focar na verificação do estado resultante. Mocks capturam interações, sendo indispensáveis quando o comportamento a ser testado é um efeito colateral sem retorno verificável — como o envio de um e-mail.

Juntos, esses padrões estabelecem um princípio essencial: testes devem ser isolados (sem dependências reais), legíveis (setup mínimo e declarativo) e focados (um único comportamento por teste). Ao construir a suíte com essa disciplina, prevenimos Test Smells sistematicamente e reduzimos o custo de manutenção a longo prazo, pois mudanças na implementação afetam apenas os Builders e Doubles — nunca a lógica dos testes em si.